CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 6 & 7 B-Trees

ملخص الميد · CLRS Chapter 18

محارب

محتوى الملف

01 وش هو الـ B-tree؟

02 الحدود: كم مفتاح في العقدة؟

03 العمليات وتعقيدها

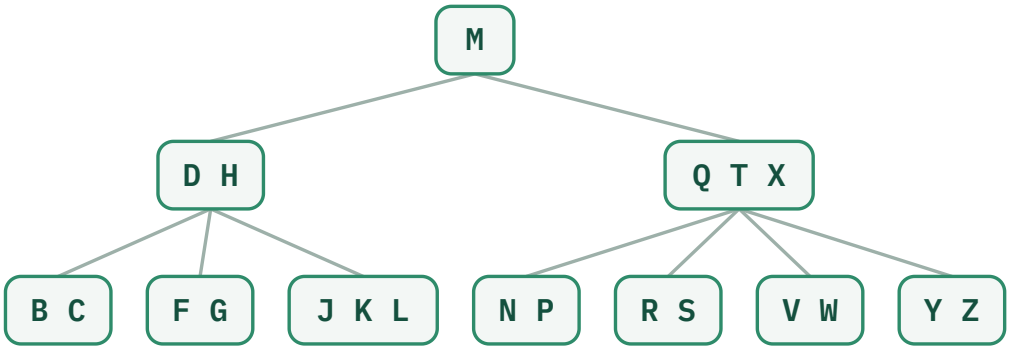
04 الإضافة Insert

05 تمرين السلايدات: t = 3

06 الحذف Delete

01 وش هو الـ B-tree؟

- شجرة بحث **متوازنة**، كل عقدة فيها أكثر من مفتاح وأكثر من ابنين.
- **كل الأوراق على نفس العمق** (= ارتفاع الشجرة h).
- لو العقدة فيها `n` مفتاح، يكون لها `n + 1` ابن.
- المفاتيح داخل العقدة مرتبة تصاعدي (non-decreasing).
- مصمم للتخزين على الأقراص (magnetic disks / secondary storage). اسمه كذلك **Balanced Sort Tree**.



B-tree	Binary tree
تخزين على القرص	تخزين في الـ RAM
أكثر من ابنين	ابنين بالكثير
متوازنة دايماً	مو لازم

02 الحدود: كم مفتاح في العقدة؟

السلايدات تستخدم **minimum degree** `t ≥ 2` :

	أقل	أكثر
مفاتيح عقدة غير الجذر	`t - 1`	`2t - 1`
أبناء عقدة داخلية غير الجذر	`t`	`2t`
الجذر	مفتاح واحد	`2t - 1` مفاتيح

```
t = 2:  1 to 3 keys,  2 to 4 children   (2-3-4 tree)
```

```
t = 3:  2 to 5 keys,  3 to 6 children
```

العقدة اللي فيها `2t - 1` مفتاح اسمها **full**.

انتبه للتسمية في الميد

الميدات تكتب «B-tree of degree 3» أو «of order 5» وتقصد **أكبر عدد أبناء** `m` : يعني أكثر شي `m - 1` مفتاح. degree 3 = مفتاحين بالكثير (2-3 tree)، و order 5 = أربع مفاتيح بالكثير. هذا غير الـ `t` اللي في السلايدات.

03 العمليات وتعقيدها

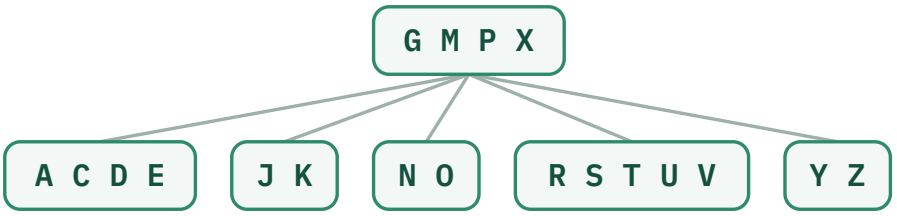
العملية	الفكرة	التعقيد
B-TREE-SEARCH(x, k)	داخل العقدة نمشي لين `k ≤ key`، لو لقيناه نرجعه، وإلا ننزل للابن المناسب	`O(t)` لكل عقدة · `O(log_t n)` عقد
B-TREE-CREATE(T)	عقدة فاضية وورقة، وتصير الجذر	`O(1)` disk و `O(1)` CPU
B-TREE-INSERT	الإضافة دايماً في ورقة	`O(t log_t n)`
B-TREE-DELETE	من أي عقدة، مع المحافظة على الحد الأدنى	`O(t log_t n)`

04 الإضافة Insert

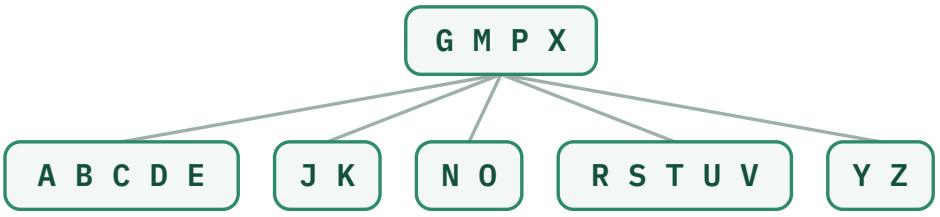
- ما نسوي ورقة جديدة؛ نضيف المفتاح في ورقة موجودة.
- لو العقدة full (`2t − 1`): نسوي **split** حول المفتاح الأوسط: الأوسط يطلع للأب، والباقي يتقسم لعقدتين كل وحدة `t − 1`.
- لو الأب full بعد، نقسمه هو كمان، ولين الجذر. لو انقسم الجذر يزيد ارتفاع الشجرة 1 (الشجرة تكبر من فوق).
- طريقة السلايدات (CLRS): نقسم أي عقدة full نمر عليها **واحنا نازلين**، قبل ما نوصل للورقة (single pass).

مثال السلايدات (t = 3): نضيف B, Q, L, F

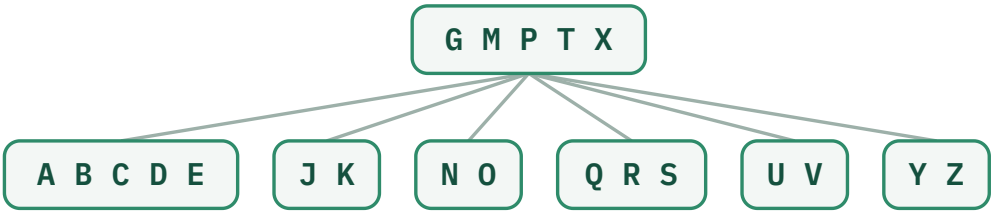
البداية:



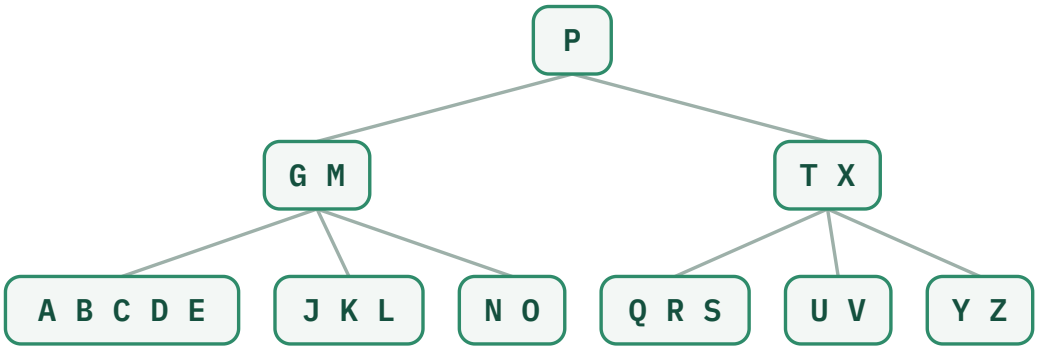
(b) B: الورقة `A C D E` مو full، يدخل مباشرة.



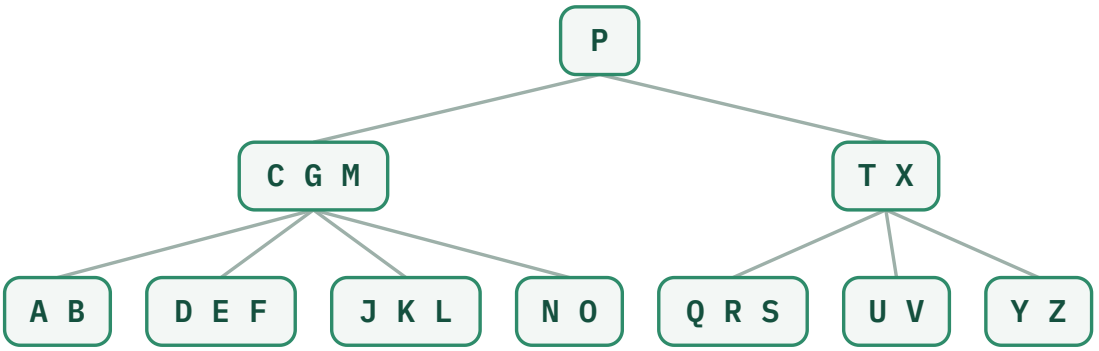
(c) Q: الورقة `R S T U V` :full → split `T` يطلع للجذر، وبعدها يدخل Q في `Q R S` .



(d) L: الجذر `G M P T X` :full → split الجذر: `P` يصير جذر جديد والارتفاع يزيد. بعدها L يدخل `J K L` .



(e) F: الورقة `A B C D E` :full → split `C` يطلع، و F يدخل `D E F` .



05 تمرين السلايدات: t = 3

```
Insert: 3, 7, 9, 23, 45, 1, 5, 14, 25,
```

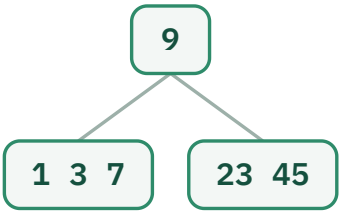
```
        24, 13, 11, 8, 19, 4, 31, 35, 56
```

```
t = 3  →  max 5 keys per node,  median = 3rd key
```

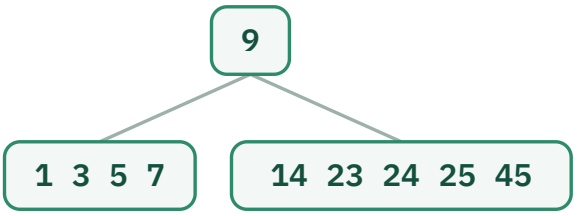
3, 7, 9, 23, 45 تدخل في ورقة وحدة وتصير full:



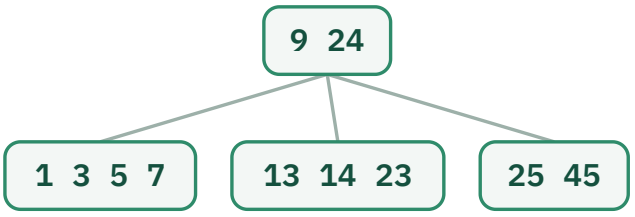
1: الجذر split → full حول 9:



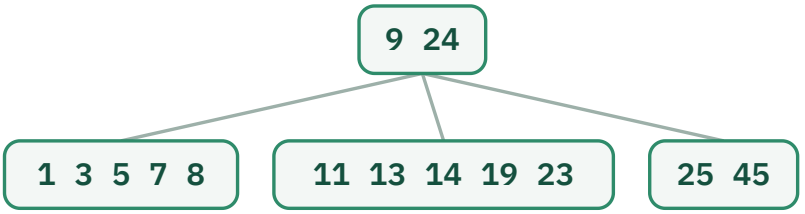
5، 14، 25، 24 تدخل بدون split:



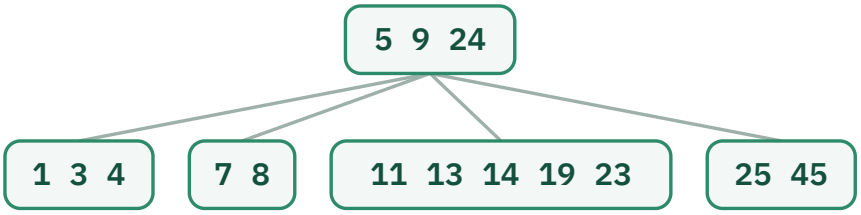
13: الورقة `14 23 24 25 45` 24 → full يطلع:



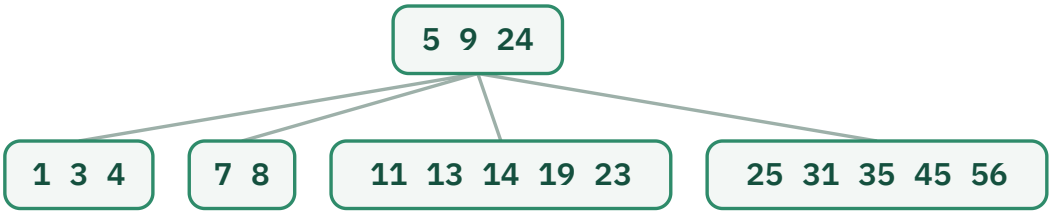
11، 8، 19 بدون split:



4: الورقة `1 3 5 7 8` 5 → full يطلع:



31، 35، 56 بدون split. الشكل النهائي:

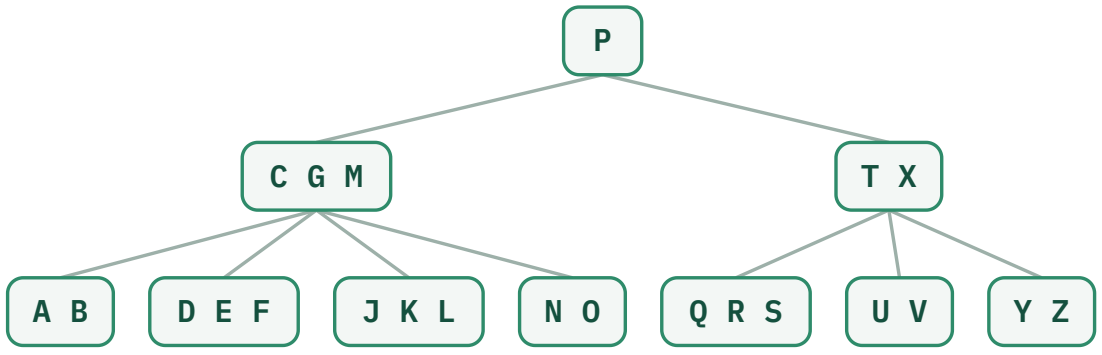


06 الحذف Delete

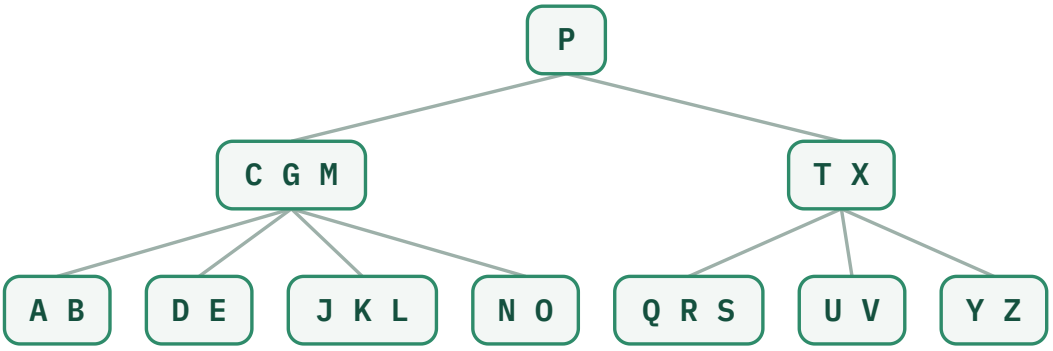
نقدر نحذف من أي عقدة، بس لازم ما تنزل أي عقدة (غير الجذر) تحت `t - 1`. الحل: ندمج العقد أو نستلف من الأخ.

الحالة	متى	وش نسوي
1	المفتاح في ورقة فيها `≥ t` مفاتيح	نحذفه وبس
2a	المفتاح في عقدة داخلية، والابن اللي قبله `≥ t`	نحط مكانه الـ **predecessor** (أكبر مفتاح في الابن اليسار) ونحذفه من تحت
2b	الابن اللي بعده `≥ t`	نحط مكانه الـ **successor** (أصغر مفتاح في الابن اليمين)
2c	الابنين كل واحد `t - 1`	**merge**: الابن اليسار + المفتاح + الابن اليمين في عقدة وحدة، ونحذف المفتاح منها
3	المفتاح مو في العقدة الحالية، والابن اللي بننزل له فيه `t - 1` بس	نكبّره قبل ما ننزل: **3a**: أخوه فيه `≥ t` → نستلف: مفتاح من الأب ينزل، ومفتاح من الأخ يطلع للأب **3b**: الأخوان كلهم `t - 1` → merge مع أخ ومفتاح من الأب

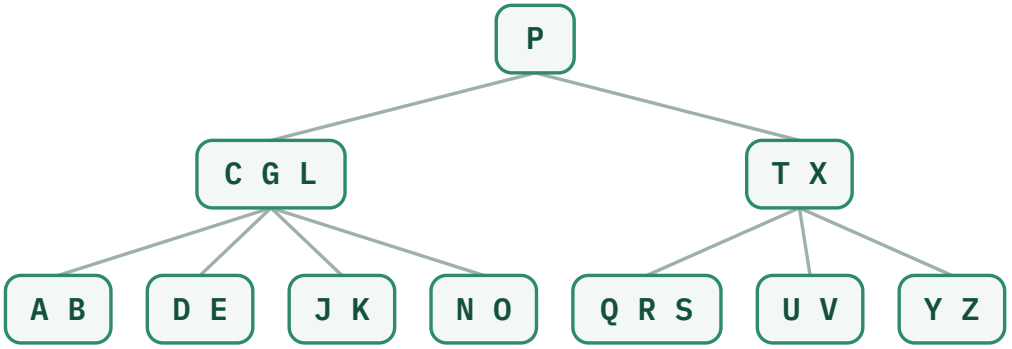
مثال السلايدات (t = 3)



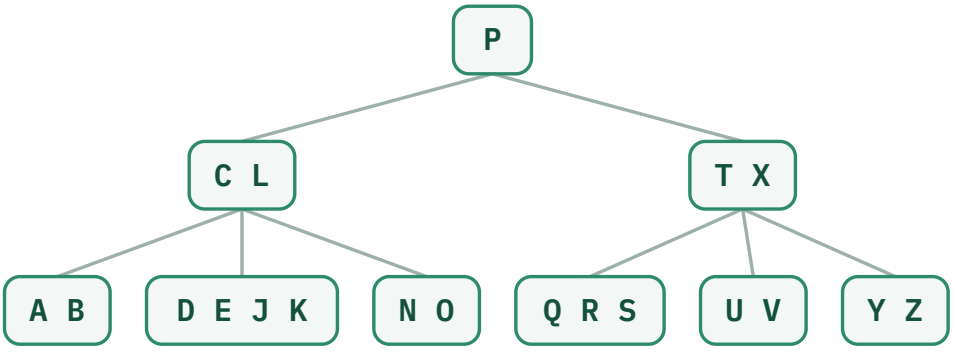
Case 1، نحذف F: الورقة `D E F` فيها $t \le 3$:



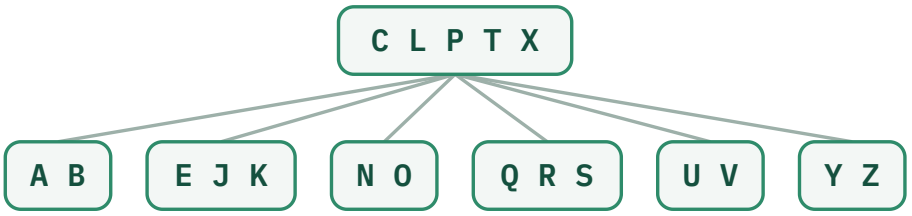
Case 2a، نحذف M: الابن اليسار `J K L` فيه 3 → L يطلع مكان M:



Case 2c، نحذف G: `D E` و `J K` كل واحد 2 → merge إلى `D E G J K` ونحذف G:



Case 3b، نحذف D: نازلين من `P` للابن `C L` وفيه 2 فقط، وأخوه `T X` بعد 2 → merge الاثنين مع P، والجذر القديم يختفي (الارتفاع ينقص). بعدها نحذف D من الورقة:



Case 3a، نحذف B: الورقة `A B` فيها 2، وأخوها `E J K` فيه 3 → C ينزل للورقة و E يطلع مكانه:

